

Learning to Detect and Describe a Wireframe

Iván Ferre^{1,2}, Luis Baumela¹, and Iago Suárez^{2,3}

¹ Universidad Politécnica de Madrid

² Machine Learning Circle

³ Qualcomm XR Labs Europe

Abstract. Image matching is essential in computer vision for applications like pose estimation, object retrieval, 3D Reconstruction, and SLAM, which typically involve Front-End (detection/description), Middle-End (matching/filtering), and Back-End (integration) stages, all constrained by computational budgets. For efficient on-device inference, sparse Front-End methods represent images as keypoints and lines forming wireframes; however, these elements are typically detected and processed independently.

We propose a unified neural network that directly produces wireframes with discriminative descriptors for each node (keypoint/junction). This unified approach enhances the efficiency of the Front-End by simultaneously finding wireframe elements and their node descriptors in a single pass, thereby reducing computational cost and simplifying the pipeline compared to traditional multi-stage methods. Our model, trained via multitask distillation from state-of-the-art point and line feature extractors, achieves a competitive trade-off between accuracy and efficiency.

Source code: <https://github.com/iferfra/wireframe-detector>

1 Introduction

Image correspondence is a core part of many computer vision applications, including pose estimation, object retrieval, 3D reconstruction, and SLAM. These applications rely on the accurate detection and matching of keypoints and line segments across images, which is critical to understanding the geometric and structural relationships of a scene. These geometric relationships are crucial in the subsequent stages of the scene understanding pipeline to identify structures such as walls, floors, ceilings, etc., or to build meaningful 3D representations such as 3D polygonal meshes.

Traditional methods for feature detection such as SIFT [12] and ORB [17] for points and LSD [22] for lines led to the rapid development of 3D vision. These sparse techniques allow for the localization and reconstruction of distinctive elements in the scene that can be accurately localized and triangulated. In modern times, new approaches based on Convolutional Neural Networks (CNNs) [3,27,24] have taken over thanks to their ability to better detect and describe under drastic changes in viewpoint and lighting.

Alongside sparse feature detection, image-to-image matching has emerged as a powerful paradigm, particularly in scenarios requiring dense pixel-level correspondences. Approaches such as LoFTR [20] and RoMa [5] leverage transformer-based architectures to establish robust dense correspondences. However, these methods often imply high computational costs, making them impractical for real-time or on-device applications. As a result, sparse techniques remain a crucial alternative, offering a more lightweight representation while still enabling effective scene reconstruction and localization.

Despite these advancements, several challenges remain. Existing methods often operate independently for keypoint and line detection, leading to redundant computations and inefficiencies. A good example is GlueStick [15], which proposes a powerful wireframe matching Middle-End but constructs the matchable wireframe by heuristically merging detections from SuperPoint [3] and LSD [22]. This highlights the need for a unified neural-based approach capable of simultaneously detecting and describing both keypoints and line segments for enhanced precision and computational efficiency.

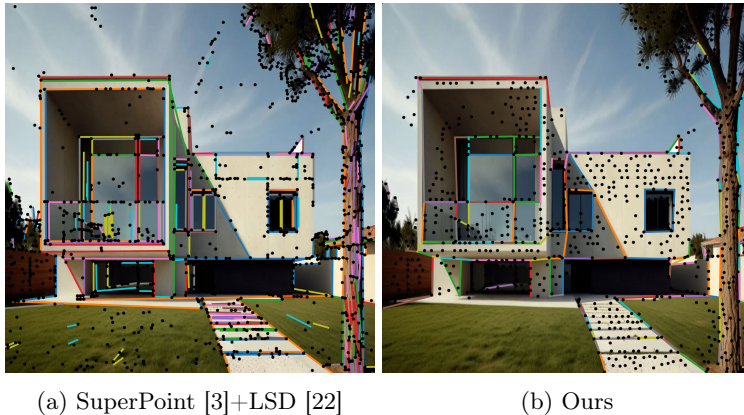


Fig. 1: **Comparison of the state of the art matchable wireframe from GlueStick (SuperPoint + LSD) vs. the result of our multitask network.** We generate semantically meaningful wireframes with cleaner lines and fewer points in unreliable textures like the sky.

We propose a new model capable of simultaneously predicting both keypoints and line segments at once, which defines a wireframe representation of the image (see Figure 1b). A wireframe is a structured representation, where keypoints act as junctions and line segments define their geometric relationships, forming a graph-like abstraction of the scene. This paper focuses on combining the capabilities of state-of-the-art techniques into a single model to streamline the feature detection process.

The architecture follows an encoder-decoder structure, where the encoder is composed of multiple convolutional layers with downsampling operations and skip connections, enabling hierarchical feature extraction. This design, initialized with pre-trained weights from DISK [21], facilitates transfer learning and accelerates convergence. The decoder consists of two specialized branches: one that predicts line segments using Holistic Attraction Field Maps (HAFM) and detects junctions, and another that extracts keypoints and their descriptors. Once the lines, keypoints, and point features are extracted, line descriptors can be computed in a post-processing step. For example, GlueStick [15] generates line descriptors by concatenating the point descriptors at the segment endpoints, ensuring a shared feature space between lines and keypoints. These initial descriptors are then enriched with positional encoding and refined through a graph neural network. The same technique can also be applied on top of our method.

We train the model using a multitask distillation framework that integrates knowledge from two independent teacher networks, one for lines and one for keypoints. Instead of relying solely on final outputs, the student learns from intermediate feature representations, enabling more efficient knowledge transfer. The training minimizes the discrepancy between the student and teacher feature maps via a weighted L_2 loss, balancing contributions from each task and promoting both robustness and computational efficiency.

We evaluate our method by comparing its performance to the teacher models and state-of-the-art methods on the HPatches and Wireframe datasets. We assess the repeatability of our detections and the quality of our matches. The proposed model is also analyzed in terms of inference speed and parameter efficiency, highlighting improvements over the individual teacher networks and demonstrating the benefits of the multitask distillation approach.

In summary, our main contributions are:

- We propose the first model that jointly detects and describes image wireframes.
- We reduce the number of parameters and execution time by 30% compared to comparable state-of-the-art models.
- We introduce a new distillation objective and procedure that combines the strengths of a line detector and a keypoint extractor. To this end, we explore different teacher-student configurations, refine the knowledge transfer strategy, test alternative ways to decode feature maps, adjust the loss weighting to balance both tasks, and modify the decoder architecture.

2 Related work

Classical corner [6,18,16,17], blob [12,2], and line detectors [4,22,19] directly use image intensities or their gradients to detect distinctive features. Modern approaches leverage deep learning to improve robustness and semantic awareness.

SuperPoint [3] uses a single encoder to detect corners and generate descriptors, training on synthetic data and refining on real images via homographic adaptation. DISK [21] trains a U-Net end-to-end with reinforcement learning,

rewarding keypoints that improve matching. ALIKED [27] pairs an encoder with a lightweight decoder and uses deformable convolutions along with sub-pixel keypoint supervision (via a Neural Reprojection Error) to compensate for reduced depth. Deformable convolutions run fast with a custom CUDA implementation but are hard to port.

In the domain of line segments, local methods such as LSD [22] or ELSSED [19] offer fast and accurate alternatives to the classical Hough Transform [4]. However, they ignore scene semantics, suffer from viewpoint inconsistencies, and are sensitive to lighting variations.

The Wireframe Dataset [8], comprising thousands of manually labeled wireframe images, has paved the way for fully supervised networks that overcome these issues. HAWP (Holistically-Attracted Wireframe Parsing) [24] detects line segments and junctions to create structured wireframes. HAWP uses a Holistic Attraction Field Map to represent segments in a 4D parameter space and uses a multi-stage pipeline to initialize, refine, and verify the candidate detections. LETR [23] directly regresses line segments with a transformer-based architecture. To remove the dependency on manual annotations, SOLD² [14] and DeepLSD [13] adopt self-supervised learning objectives based on homography adaptation.

Parallel to detection, various methods have been developed to describe line segments for triangulation and reconstruction. LBD [26] characterizes lines with histograms of gradient orientations from surrounding textures, while LDL [9] leverages deep learning for a similar purpose. SOLD² [14] takes a different approach, predicting a dense feature map and sampling point descriptors along the line to form the line descriptor. LineTR [25] further condenses this information using a visual transformer to generate compact descriptors. Lastly, GlueStick [15] demonstrates that simply concatenating the descriptors at a line’s endpoints, combined with robust matching strategies to handle occlusions, offers a practical solution that outperforms all the region-based predecessors.

However, methods like GlueStick rely on separately detecting keypoints and lines, requiring heuristic merging strategies that may introduce inconsistencies. A unified multitask model is needed not only to streamline feature extraction but also to ensure a more coherent representation, where keypoints and lines are detected and described in a structurally consistent manner.

3 Method

The proposed method maximizes computational reuse across keypoint detection, line detection, and keypoint extraction. To achieve this, we use a hybrid architecture with a shared encoder and two decoders: one for points and one for lines (Fig. 2). Given an input image I , the shared encoder, composed of multiple convolutional layers with downsampling operations and skip connections, produces a feature map $\mathcal{F}_{\text{shared}} \in \mathbb{R}^{H \times W}$, which represents the content of the image and is the input to both decoders. In the following, we explain how we can extract the wireframe composed of lines, points, and descriptors from $\mathcal{F}_{\text{shared}}$.

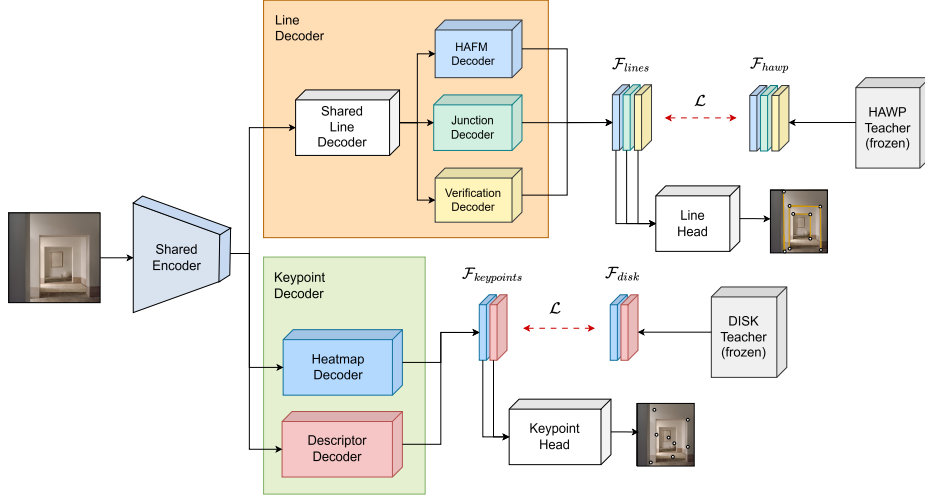


Fig. 2: **Proposed architecture for joint line and point detection and description.** The model follows an encoder-decoder structure with a shared encoder pre-trained from DISK and two decoder branches: one for line segments using Holistic Attraction Field Maps (HAFM), junctions, and verification features, and another for keypoints and their descriptors. Each branch has a dedicated prediction head, and training uses the distillation loss $\mathcal{L}$ to align the student’s feature maps with those of the teacher models (HAWP for lines and DISK for keypoints).

3.1 Line Holistic Attraction Field Map

We aim to train our neural network to detect line segments from images. However, directly predicting a simple and geometrically meaningful representation is challenging. Instead, a per-pixel vector field is more suitable for learning. Therefore, we represent lines as a 4-D Holistic Attraction Field Map (HAFM) [24], which provides an exact, closed-form dual representation of line segments. Consider a line segment $\vec{l} = (\mathbf{x}_1, \mathbf{x}_2)$, $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^2$, with its supporting line defined by $l : \mathbf{a}_l^\top \mathbf{x} + b_l = 0$. For any point $\mathbf{p} \in \mathcal{A}$ (the HAFM) that is attracted to $\vec{l}$, let $d > 0$ denote the Euclidean distance from $\mathbf{p}$ to l . We then apply a similarity transformation to $\mathbf{p}$ consisting of:

1. **Translation:** Move $\mathbf{p}$ to the origin.
2. **Rotation:** Rotate the coordinate system such that l aligns with the vertical axis, introducing a rotation angle $\theta \in [-\pi, \pi)$.
3. **Scaling:** Normalize the coordinate axes by the distance d .

This transformation yields two intrinsic angular parameters, $\theta_1 \in (0, \frac{\pi}{2})$ and $\theta_2 \in (-\frac{\pi}{2}, 0]$, which capture the orientations of the endpoints relative to $\mathbf{p}$. Thus, we re-parameterize each point $\mathbf{p}$ in the support region of $\vec{l}$ as $\mathbf{p}(\vec{l}) = (d, \theta, \theta_1, \theta_2)$.

In practice, we set a distance threshold $d_{\max}$, filtering out all points in $\mathcal{A}$ if their d 's are greater than $d_{\max}$ by labeling them as "background" with vector $(-1, 0, 0, 0)$. This compact 4-D representation effectively decouples the geometric attributes of the line segment, thereby facilitating efficient learning with deep neural networks while ensuring that the original line segment $\tilde{l}$ can be recovered exactly in closed form.

3.2 Line Segment Decoder

The entire pipeline to extract line segments from the image is composed of several key steps. The line head takes as input the shared features $\mathcal{F}_{\text{shared}}$. From them, a line-specific decoder, consisting of three convolutional layers (two 3×3 with strides of 2 and one 1×1), along with batch normalization after the first layer, extracts intermediate line features $\mathcal{F}_{\text{shared-lines}}$ from which we predict:

1. The 4-D Holistic Attraction Field Map

$$\hat{\mathcal{A}} = \sigma\left(f_{\theta}^{(1)}(\mathcal{F}_{\text{shared-lines}})\right), \quad (1)$$

where $\sigma(\cdot)$ denotes the sigmoid activation and f_{θ} is parametrized as a small CNN composed of three convolutional layers: a 3×3 layer followed by batch normalization to refine spatial features, a 1×1 layer to reduce the number of channels, and a final 1×1 layer that outputs the predicted feature maps. The superscript in $f_{\theta}^{(i)}$ indicates different output branches from the same network.

2. A distance residual map $\hat{\Delta}d$ designed to make small corrections in the estimation of $\hat{\mathcal{A}}$:

$$\hat{\Delta}d = \sigma\left(f_{\theta}^{(2)}(\mathcal{F}_{\text{shared-lines}})\right). \quad (2)$$

This is important because among the 4 dimensions to predict, d is the most unstable one. During inference, the refined distance is computed as

$$\hat{d}'(\kappa) = \hat{d} + \kappa \hat{\Delta}d, \kappa \in \{-1, 0, 1\}, \quad (3)$$

and proposals are generated if $0 < \hat{d}'(\kappa) \leq d_{\max}$.

3. A junction map and an offset map

$$\hat{\mathcal{J}} = \sigma(h_{\theta}(\mathcal{F}_{\text{shared-lines}})), \hat{\mathcal{O}} = \sigma(h_{\theta}(\mathcal{F}_{\text{shared-lines}}) - 0.5), \quad (4)$$

where h_{θ} is a lightweight CNN with a structure similar to f_{θ} . We apply a 3×3 non-maxima suppression (NMS) to $\hat{\mathcal{J}}$, retaining the top- K junctions at locations $\hat{\mathbf{x}}_b$, and each junction's position is computed as $\hat{\mathbf{x}}_j = \hat{\mathbf{x}}_b + w \cdot \hat{\mathcal{O}}(\hat{\mathbf{x}}_b)$, where w is a rescaling factor. The result is a list of 2D points $\hat{\mathbf{x}}_j = [\hat{\mathbf{x}}_j^1, \dots, \hat{\mathbf{x}}_j^K]$.

4. The verification features (LOI [28] features) $\hat{\mathcal{L}} = \text{Conv}_{1 \times 1}(\mathcal{F}_{\text{shared-lines}})$, used to score the quality of proposed segments.

To propose line segments from these outputs a line segment proposal is kept if both its endpoints are within a Euclidean distance τ (e.g., $\tau = 10$) of a corresponding junction proposal. Junctions not matched to any line segment are discarded.

Lastly, for each proposed line segment $\vec{l}$, a fixed-size feature vector is extracted from the verification features $\hat{\mathcal{L}}$ using a LoIPool [28] operation that uniformly samples s points (with $s = 32$) along $\vec{l}$. The extracted features are concatenated and passed through a two-layer fully connected classifier, which outputs a score representing the quality of the segment.

Unlike HAWP [24], which refines features using 1×1 convolutions, our model employs multiple 3×3 convolutions with batch normalization to better separate keypoint and line features and capture geometric structure through larger receptive fields.

3.3 Keypoint Detection and Description Decoder

In parallel with the line extraction process, a dedicated head is responsible for identifying keypoints in the image. This module takes the shared features $\mathcal{F}_{\text{shared}}$ as input and processes them to produce:

1. A heatmap $\hat{\mathcal{H}}$ encoding keypoint locations, where each value represents the likelihood of a keypoint being present at that location:

$$\hat{\mathcal{H}} = \sigma(f_{\theta}(\mathcal{F}_{\text{shared}})), \quad (5)$$

where f_{θ} is a convolutional layer with a 3×3 kernel and a single output channel.

2. A descriptor field $\hat{\mathcal{D}}$, where each spatial location stores a feature vector describing the local image content:

$$\hat{\mathcal{D}} = \text{normalize}(f_{\theta}(\mathcal{F}_{\text{shared}})), \quad (6)$$

where $\text{normalize}(\cdot)$ applies L2 normalization, scaling each descriptor to the unit norm as $\mathbf{x}/\|\mathbf{x}\|_2$. The descriptor decoder f_{θ} consists of a sequence of four convolutional layers: three 3×3 convolutions followed by a 1×1 convolution for channel reduction. The first convolution is followed by a batch normalization layer to stabilize training. This descriptor field provides a dense per-pixel representation of the image, which is later sampled at specific keypoint locations.

To extract the final keypoint set, we apply NMS over $\hat{\mathcal{H}}$, selecting the top- K local maxima as candidate points. Additionally, the junctions extracted from the line head are incorporated into the final set of keypoints, preserving structural information from line intersections. The final set of keypoints is thus given by $\hat{\mathbf{x}}_{\text{final}} = [\hat{\mathbf{x}}_h, \hat{\mathbf{x}}_j]$, where $\hat{\mathbf{x}}_h$ represents the detected keypoints from the heatmap, and $\hat{\mathbf{x}}_j$ corresponds to the junctions extracted from the lines head. Once the keypoints are determined, we extract their corresponding feature descriptors by bilinear sampling $\hat{\mathcal{D}}_{\text{final}} = \hat{\mathcal{D}}(\hat{\mathbf{x}}_{\text{final}})$.

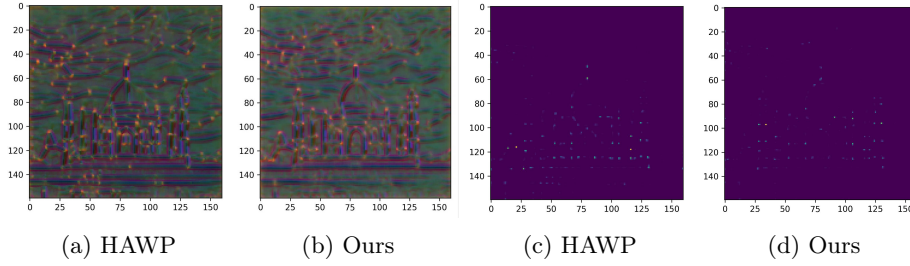


Fig. 3: **Comparison of the Holistic Attraction Field Map (HAFM) and junctions heatmap between HAWP (teacher) and our model (student).** The left two columns (a, b) show a PCA visualization of the HAFM, where we observe that the feature map produced by the student closely resemble these of the teacher, indicating effective knowledge transfer. The right two columns (c, d) display the junction heatmaps from both models, demonstrating that the student successfully replicates the junction localization patterns learned from the teacher.

3.4 The Multitask Distillation Framework

We introduce a unified multitask distillation framework for simultaneous keypoint and line segment detection and description. Our method distills from two teachers, HAWP [24] for line detection, and DISK [21] for keypoint detection, both open-source models with permissive license. We train a compact student model, reducing redundancy, and ensuring consistent feature representations. This makes it ideal for real-time, high-accuracy applications such as robotics, augmented reality, and 3D reconstruction.

The approach uses a shared encoder-decoder architecture. We initialize the encoder with DISK’s pre-trained weights. Next, each decoder specializes in one task. Task-specific heads refine these outputs into the final predictions, achieving performance on par with the teacher models while significantly reducing computational complexity. HAWP provides expertise in detecting line segments and their connections as graphs, while DISK excels at identifying keypoints and generating robust descriptors. Through knowledge distillation, the student model inherits the strengths of both teachers, ensuring a seamless and unified detection process.

Knowledge distillation [7] allows learning from a large, complex model (the teacher) to a smaller, more efficient model (the student). The student learns by minimizing the differences between its feature maps and those of the teachers. We display an example of teacher and student feature maps in Figure 3. This is achieved using an L_2 loss function, defined as

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left\| \mathcal{F}_i^{\text{teacher}} - \mathcal{F}_i^{\text{student}} \right\|^2, \quad (7)$$

where $\mathcal{F}_i^{\text{teacher}}$ and $\mathcal{F}_i^{\text{student}}$ represent the feature maps of the teacher and student models, respectively. We instantiate this loss in several tensors, each one targeting a different task. $\mathcal{L}_{\text{keypoints}}$ and $\mathcal{L}_{\text{descriptors}}$ compare the keypoint heatmap predictions $\hat{\mathcal{H}}$ and the descriptor predictions $\hat{\mathcal{D}}$ with the teacher’s ones. On the lines side, $\mathcal{L}_{\text{hafm}}$, $\mathcal{L}_{\text{verification}}$, and $\mathcal{L}_{\text{junctions}}$ compare the predictions for the Holistic Attraction Field Map $\hat{\mathcal{A}}$, the verification features $\hat{\mathcal{L}}$, and the joints heatmap $\hat{\mathcal{J}}$ respectively. To balance the contributions of different tasks, the loss function assigns weights to each component. The final loss function is

$$\mathcal{L}_{\text{total}} = 0.4 \cdot \mathcal{L}_{\text{keypoints}} + 0.2 \cdot \mathcal{L}_{\text{descriptors}} + 0.4 \cdot \mathcal{L}_{\text{hafm}} + 0.1 \cdot \mathcal{L}_{\text{verification}} + 0.3 \cdot \mathcal{L}_{\text{junctions}} \quad (8)$$

These weights were determined empirically to ensure that the student model learns effectively from both teacher models without favoring one task over the other. Dynamic Weight Averaging (DWA) [11] was also explored as an alternative for assigning weights to the loss components dynamically. However, manually tuned weights provided slightly better performance.

4 Experiments

This section presents the datasets, metrics, and key results for evaluating the multitask model. We compare the multitask model against its teacher models (HAWP and DISK), reporting both quantitative results and qualitative visualizations.

4.1 Training Details

The multitask model is implemented using PyTorch Lightning and trained for 35 epochs until convergence, using a combined dataset of COCO [10] and Wireframe (train split) [8], containing approximately 13,000 examples per epoch. We used Adam optimizer with an initial learning rate of 1×10^{-4} , which was reduced whenever the validation loss plateaued. The model took around 12 hours to train on an NVIDIA A100 GPU.

4.2 Quantitative Results

The HPatches [1] dataset was used to evaluate keypoint detection and descriptor matching. It is a standard benchmark for local feature evaluation, comprising sequences of real-world planar scenes with known ground-truth homographies, designed to test robustness under significant changes in viewpoint and illumination.

To ensure fair evaluation, only points visible in both images of each pair were considered, using the provided homographies to filter valid regions. Several metrics were employed:

- **Repeatability**: Measures the fraction of keypoints consistently detected in both images, evaluated at distances of 1 and 3 pixels.

Model	# Points	Repeatability		Correct Matches			mAP
		1px	3px	1px	3px	10px	
SIFT	3212	0.262	0.539	0.299	0.406	0.443	0.627
SuperPoint	1922	0.235	0.599	0.284	0.417	0.472	0.524
ALIKED	1628	0.330	0.616	0.298	0.425	0.474	0.604
DISK	4232	0.249	0.650	0.304	0.438	0.476	0.579
Ours	1454	0.301	0.500	0.278	0.427	0.474	0.567

Table 1: Comparison of keypoint repeatability, ratio of correct matches, number of detected keypoints and mAP across different models on the HPatches dataset.

- **Correct Matches:** Assesses the percentage of correctly matched keypoints at various distance thresholds (1, 3, and 10 pixels).
- **mAP (Mean Average Precision):** Measures the quality of keypoint matching by evaluating precision across different error thresholds. It computes how many correct matches are ranked higher, by averaging precision over these thresholds. This helps evaluate both accuracy and consistency in keypoint matching.
- **f1@Npx:** It balances precision and recall, measuring both the accuracy and completeness of detected matches. It is calculated as the harmonic mean of precision and recall, ensuring a trade-off between detecting correct matches and avoiding false positives.
- **msAP (Mean Structural Average Precision)** [24]: Measures the quality of line segment detection by averaging sAP across multiple distance thresholds. It considers both the geometric accuracy and confidence scores of predicted lines, computing the area under the precision-recall curve at different tolerance levels to provide a comprehensive evaluation.

We evaluate the model’s line detection performance on the HPatches and Wireframe (test split) datasets. The Wireframe dataset consists of real-world images annotated with manually labeled line segments and junctions, capturing man-made environments with strong geometric structures.

The main metric, Mean Structural Average Precision (msAP), measures how well the model predicts line segments compared to ground truth. It evaluates precision and recall across varying tolerance levels, providing a global measure of performance.

To evaluate efficiency, the inference time and parameter count were measured for each model separately (HAWP and DISK), their combined total, and the multitask model. The measurements were conducted using a NVIDIA GeForce GTX 1070 GPU with images of size 640x640. The inference time was averaged over 1,000 images for consistency.

The results in Tables 1–3 highlight the efficiency and competitive performance of the multitask model. As shown in Table 1, the model achieves higher repeatability at 1px, while at 3px, there is a slight decrease. The percentage of

Models	HPatches				Wireframe			
	# Lines	f1@1px	f1@3px	msAP	# Lines	f1@1px	f1@3px	msAP
LSD	424	0.139	0.253	0.272	332	0.219	0.454	0.290
HAWP	773	0.133	0.226	0.329	841	0.175	0.476	0.221
Ours	670	0.140	0.220	0.306	755	0.169	0.414	0.243

Table 2: Comparison of line metrics on the HPatches and Wireframe datasets.

Models	Inference Time (ms)	# Parameters
Sum of Teacher Models	167	12.822.865
Ours	117	5.634.960

Table 3: Comparison of inference time and parameter count between the teacher models and the proposed multitask model.

correct matches at 1px is marginally lower, but performance remains comparable to state-of-the-art methods, particularly at larger thresholds. The mAP score is also similar to DISK.

Table 2 demonstrates that our model achieves a strong balance between precision and efficiency in line detection. While it detects fewer lines than HAWP, it maintains a similar f1@1px and f1@3px while achieving a higher msAP on the Wireframe dataset, indicating improved structured representation.

Most importantly, Table 3 shows a substantial reduction in computational cost. The multitask model reduces inference time by 30% and the number of parameters by over 7 million compared to the combined teacher models. These improvements make it a practical solution for real-time applications requiring both keypoint and line detection in resource-constrained environments.

In Figure 4, we present examples of keypoints and line segments detected by both the teacher and student models.

5 Conclusion

We presented the first unified neural network architecture that jointly detects and describes image wireframes by seamlessly integrating keypoint and line segment extraction into a single, efficient framework. This integration offers both advantages and challenges in combining multiple tasks into a single model.

By leveraging a novel multitask distillation strategy, we transfer knowledge from state-of-the-art teacher models into a compact and shared encoder-decoder network. Our approach not only maintains competitive accuracy in terms of keypoint repeatability and matching quality, but also substantially reduces inference time and parameter count. The model’s ability to perform both tasks in parallel makes it well-suited for real-time applications, such as SLAM and

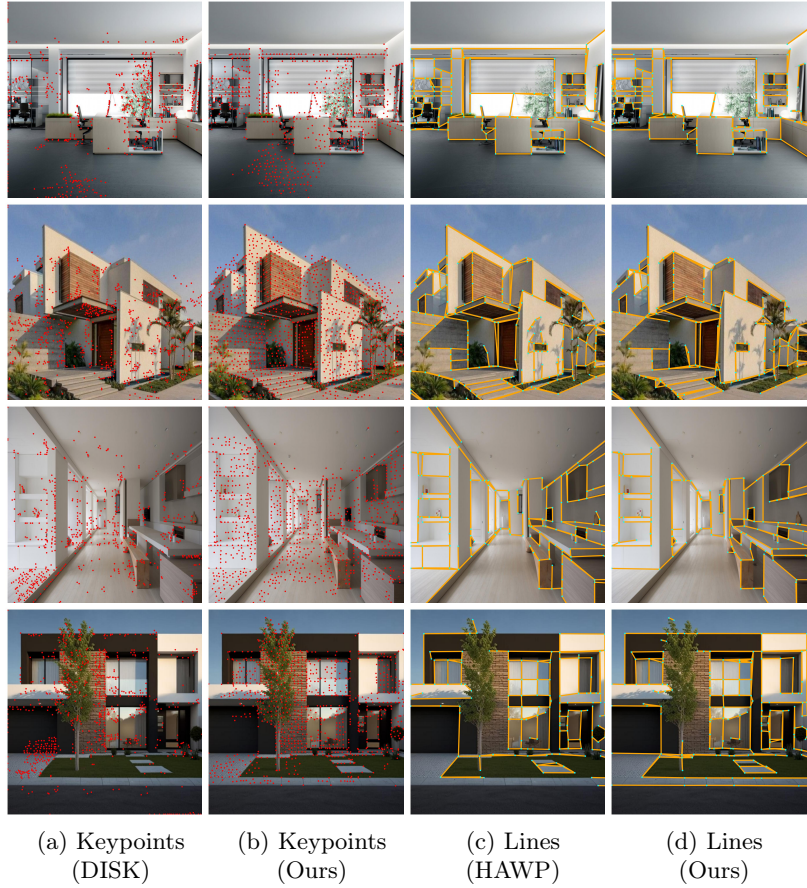


Fig. 4: **Comparative visualizations:** The left two columns show keypoints detected using DISK (left) and the trained model (right), while the right two columns show lines detected using HAWP (left) and the trained model (right).

3D reconstruction. This makes it a strong candidate for deployment in resource-constrained environments, such as mobile devices. Additionally, merging tasks typically handled independently simplifies the architecture, resulting in a more compact and efficient pipeline.

In future work, we aim to address some of the limitations of the current model by exploring the difficult task of training the network in a self-supervised way. We would like to do joint training of our model (Front-End) and GlueStick (Middle-End). Finally, we aim to explore the correlation of our wireframes with artist-made 3D assets.

In summary, our contributions confirm the feasibility of a multitask approach, pave the way toward fully differentiable wireframe matching, and provide the

community with an efficient model for extracting matchable wireframes within a single unified network.

Acknowledgments

The authors would like to thank the [ML Circle](#) and the PCR Computer Vision Lab research group at Universidad Politécnica de Madrid for their support and insightful discussions throughout the development of this work.

Luis Baumela was supported by project PID2022-137581OB-I00 funded by MCIU/AEI/10.13039/501100011033, FEDER, UE.

References

1. Balntas, V., Lenc, K., Vedaldi, A., Mikolajczyk, K.: Hpatches: A benchmark and evaluation of handcrafted and learned local descriptors. In: CVPR (2017)
2. Bay, H., Ess, A., Tuytelaars, T., Van Gool, L.: Speeded-up robust features (surf). *CVIU* **110**(3), 346–359 (2008)
3. DeTone, D., Malisiewicz, T., Rabinovich, A.: Superpoint: Self-supervised interest point detection and description. In: Proceedings of the IEEE conference on computer vision and pattern recognition workshops. pp. 224–236 (2018)
4. Duda, R.O., Hart, P.E.: Use of the hough transformation to detect lines and curves in pictures. *Communications of the ACM* **15**(1), 11–15 (1972)
5. Edstedt, J., Sun, Q., Bökman, G., Wadenbäck, M., Felsberg, M.: Roma: Robust dense feature matching. In: CVPR. pp. 19790–19800 (2024)
6. Harris, C., Stephens, M., et al.: A combined corner and edge detector. In: Alvey vision conference. vol. 15, pp. 10–5244. Citeseer (1988)
7. Hinton, G.: Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531 (2015)
8. Huang, K., Wang, Y., Zhou, Z., Ding, T., Gao, S., Ma, Y.: Learning to parse wireframes in images of man-made environments. In: CVPR (June 2018)
9. Lange, M., Schweinfurth, F., Schilling, A.: Dld: A deep learning based line descriptor for line feature matching. In: IROS. pp. 5910–5915. IEEE (2019)
10. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft coco: Common objects in context. In: ECCV. pp. 740–755 (2014)
11. Liu, S., Johns, E., Davison, A.J.: End-to-end multi-task learning with attention. In: CVPR. pp. 1871–1880 (2019)
12. Lowe, D.G.: Distinctive image features from scale-invariant keypoints. *IJCV* **60**, 91–110 (2004)
13. Pautrat, R., Barath, D., Larsson, V., Oswald, M.R., Pollefeys, M.: Deeplsd: Line segment detection and refinement with deep image gradients. In: CVPR. pp. 17327–17336 (2023)
14. Pautrat, R., Lin, J.T., Larsson, V., Oswald, M.R., Pollefeys, M.: Sold2: Self-supervised occlusion-aware line description and detection. In: CVPR. pp. 11368–11378 (2021)
15. Pautrat, R., Suárez, I., Yu, Y., Pollefeys, M., Larsson, V.: Gluestick: Robust image matching by sticking points and lines together. In: ICCV. pp. 9706–9716 (2023)

16. Rosten, E., Drummond, T.: Machine learning for high-speed corner detection. In: ECCV. pp. 430–443 (2006)
17. Rublee, E., Rabaud, V., Konolige, K., Bradski, G.: Orb: An efficient alternative to sift or surf. In: ICCV. pp. 2564–2571. Ieee (2011)
18. Shi, J., et al.: Good features to track. In: CVPR. pp. 593–600 (1994)
19. Suárez, I., Buenaposada, J.M., Baumela, L.: Elsed: Enhanced line segment drawing. *Pattern Recognition* **127**, 108619 (2022)
20. Sun, J., Shen, Z., Wang, Y., Bao, H., Zhou, X.: Loftr: Detector-free local feature matching with transformers. In: CVPR. pp. 8922–8931 (2021)
21. Tyszkiewicz, M., Fua, P., Trulls, E.: Disk: Learning local features with policy gradient. *NIPS* **33**, 14254–14265 (2020)
22. Von Gioi, R.G., Jakubowicz, J., Morel, J.M., Randall, G.: Lsd: A line segment detector. *Image Processing On Line* **2**, 35–55 (2012)
23. Xu, Y., Xu, W., Cheung, D., Tu, Z.: Line segment detection using transformers without edges. In: CVPR. pp. 4257–4266 (2021)
24. Xue, N., Wu, T., Bai, S., Wang, F.D., Xia, G.S., Zhang, L., Torr, P.H.: Holistically-attracted wireframe parsing: From supervised to self-supervised learning. *PAMI* (2023)
25. Yoon, S., Kim, A.: Line as a visual sentence: Context-aware line descriptor for visual localization. *IEEE Robotics and Automation Letters* **6**(4), 8726–8733 (2021)
26. Zhang, L., Koch, R.: An efficient and robust line segment matching approach based on lbd descriptor and pairwise geometric consistency. *Journal of visual communication and image representation* **24**(7), 794–805 (2013)
27. Zhao, X., Wu, X., Chen, W., Chen, P.C., Xu, Q., Li, Z.: Aliked: A lighter keypoint and descriptor extraction network via deformable transformation. *IEEE Trans. on Instrumentation and Measurement* **72**, 1–16 (2023)
28. Zhou, Y., Qi, H., Ma, Y.: End-to-end wireframe parsing. In: ICCV. pp. 962–971 (2019)